

# Evaluating Document Similarity Detection Approaches for Content Drift Detection

**Cite as:** Martin Paul Eve, "Evaluating Document Similarity Detection Approaches for Content Drift Detection", *eve.gd: Martin Paul Eve*, December 21, 2024, <https://doi.org/10.59348/xnrh7-7px07>.

---



“Content drift” is an important concept for digital preservation and web archiving. Scholarly readers expect to find immutable (“persisted”) content at the resolution endpoint of a DOI. It is a matter of research integrity that research articles should remain the same at the endpoint, as citations can refer to specific textual formulations.

Detecting content drift is not an easy task as it is not just a matter of seeing whether a web page has changed. HTML pages can change in layout, but retain exactly the same content in a way that is obvious to humans but not to machines. Thus, there is a requirement to parse web pages into plaintext versions; itself a difficult task due to JavaScript components and CAPTCHA systems. Then this preprocessing must discard dynamic user content, such as comments and annotations that can interfere with document similarity detection.

This post details the performance and resources required by a range of approaches to document similarity detection once these preprocessing elements have been completed. The algorithms evaluated include older systems such as Jaccard Similarity right through to modern machine learning and AI models.

The conclusion is that earlier algorithms provide a better measure of “content drift” but a worse semantic understanding of documents. However, we do not want a “semantic understanding” of document similarity in this use case. For a person “This jar is empty” and “This jar has nothing in it” are the same thing, semantically. But for the purpose of working out whether the document has changed in a scholarly context, we would say it has drifted, because someone might have cited “This jar is empty” and expects to find this formulation at the persisted endpoint. Hence, machine learning models that perceive semantic formulations as “similar” fare worse for the job of content drift detection than earlier algorithms that detect lexical change and containment.

## Background

The most well-known form of preservation challenge is “link rot”, a situation where a hyperlink no longer functions because the destination material has disappeared. A related phenomenon to this is where the content of a hyperlinked page changes such that it no longer reflects the state at the time that a link was made. This is known as “content drift”, a term initially set out by Jones et al. in 2016, and to which much of this work is indebted.<sup>1</sup> For scholarly uses cases, content drift matters because readers expect to find immutable content at the resolution endpoint of a DOI. It quickly becomes a matter of research integrity if research content changes at the endpoint. However, pages may change in layout, font, or even correct typos and still be considered “roughly” the same content.

Indeed, content drift is a difficult term to define, as there are different categories of content on web pages, some of which matter and others of which do not. For instance, if there is an advert on a web page that changes, but the main content body remains the same, has the content drifted? Likewise when a major platform redesigns their site, but retains the central content, is this content drift? The difference lies between the bitstream that is delivered to the client varying vs. the human-readable content changing. Even this binary, though, is not that clear: if a small typo is corrected, does this constitute content drift? Thus, the decision about whether something has or has not drifted is subjective.

This definitional problem recurs in existing papers on the topic. For instance, Cho and Garcia-Molina report that “by change we mean any change to the textual content of a page”, using the checksum of the page’s HTML as the marker of whether the page has changed.<sup>2</sup> Cho and Garcia-

Molina's primary finding was that "more than 40% of pages in the com domain changed every day, while less than 10% of the pages in other domains changed at that frequency" and the goal of their paper was to construct a mathematical model for change periodicity, allowing a web crawler to avoid batch refreshing its entire database on every crawl.<sup>3</sup> Yet their approach was a byte-for-byte comparison for the definition of "change", which could mean, for instance, that if someone added a comment to an article, that it would be detected as "changing". Under such a byte-wise comparison, even the slightest change (a new advert, for instance) could register as content drift.

Problematically, when adopting the byte-wise comparison methods, some (much older) studies have shown that 16.5% of websites changed on every visit, even if the two visits were immediately consecutive.<sup>4</sup> In order to avoid such problems, not all approaches to detecting content drift have been automated. Bowers et al. surveyed 4,500 papers in a "human-reviewed" sample, implying that their definition of drift cannot be the precision of the digital bytestream, but instead was looking for "the information journalists had intended".<sup>5</sup>

In short, problems of detecting content drift are a subset of document similarity detection problems.<sup>6</sup> There are several methods for detecting document similarity, which include Euclidean Distance, Cosine Similarity, Jaccard Similarity/Coefficient, and Correlation Coefficient, among many others.<sup>7</sup> This document will explore the current state of these methods and their implications for content drift detection.

## **Pre-Processing**

There are several steps of preprocessing, with each decision having knock-on effects for how well content drift will be detected, within a particular context.

### **Rendering HTML and Extracting Plaintext**

The first decision is how to render the HTML of the web page and to extract the plaintext of the landing page. The obvious solution is to use a full rendering engine of a headless browser so that the end result is as close to the user experience of a DOI resolver as possible. This may also be desirable because some publishers depend heavily on JavaScript in their platform to render even the most basic content. However, this approach has problems. First and foremost, it may mean that

dynamic user-generated content appears in the document, which will distort any similarity measures, even though the core content has not changed. Second, JavaScript may be used to implement Digital Rights Management (DRM) technologies that are designed to prohibit automated access, thereby making content drift detection virtually impossible. For instance, bypassing Cloudflare's CAPTCHA system is extremely difficult.

## **Tokenization and Word vs. Character -Level Decisions**

The next step, if using a bag of words model, is to tokenize the document. This means to decompose the document into words or character chunks. Notably, this approach discards embeddings and sequencing, so it is not necessarily possible after tokenization to read a term contextually. While character-based tokenization may perform better for small changes, they produce longer input buffers and lose human-readable context when debugging. A final type of tokenization, sometimes used in machine learning models, is subword tokenization, where known words are combined in their smallest forms; so, for example, "transformers" might be tokenized as {"transform", "ers"}.

## **Lemmatization and Standardisation**

Another decision that must be made is whether to lemmatize the document and/or word-tokens. This reduces a word to its root form. So, for instance, "running" would be reduced to "run". This would eradicate the difference between "run", "running", "runs", and other forms. This can be useful if you want the similarity scoring to be less sensitive to minor changes to the forms of words.

## **Lowercasing**

It can sometimes be beneficial to lowercase all input to reduce sensitivity to capitalisation differences ("Martin" vs "martin").

## **Removing Punctuation and Stopwords**

Sometimes, punctuation and commonly used stopwords ("and", "in" etc.) can over-sensitise a model to differences that are trivial. Consider whether removing punctuation or stopwords helps achieve the sensitivity needed. This was certainly the case with the cosine similarity coefficient.

# Methods

## Feature Detection

Among the simplest methods for content drift detection is to pick up some specific features of DOI landing pages that are part of the membership terms of the issuing registrar. For instance, detecting the DOI and the page title / list of authors is at least something that should be possible. This may allow for detection of unobvious page changes, such as domain hijacking or total content replacement. Given that displaying these data is a criterion of DOI registrars' memberships, this is actually a surprisingly powerful measure for content drift detection.

## Resolution to Open Access PDFs

In cases where a DOI resolves directly to an open-access Portable Document Format (PDF) file, it makes more sense to treat any binary change as content drift and it may be appropriate to use checksum techniques in this context. This is because the PDF is encapsulated and does not have the content divide mentioned above. If the file is replaced, it makes sense to say it has drifted, as most publishers do not redesign their PDFs while maintaining the immutable content.

*Edit:* As [Geoffrey Bilder](#) has pointed out to me, though, these files can be watermarked and/or add custom cover pages, thereby tripping up binary comparison.

## Jaccard Similarity Coefficient, Jaccard Containment, and Shingling

One of the most longstanding techniques for measuring page change that does not depend on a comparison of the exact bytestream came from Fetterly et al., who implemented a version of Broder et al.'s syntactic similarity clustering method.<sup>8</sup> The logic behind this particular approach is “a mechanism for discovering when two documents are ‘roughly the same’; that is, for discovering when they have the same content except for modifications such as formatting, minor corrections, webmaster signature, or logo”.<sup>9</sup> This informal expression of “roughly the same” is denoted by the mathematical concepts of “resemblance” and “containment”.

The “resemblance” of two documents, “A and B is a number between 0 and 1, such that when the resemblance is close to 1, it is likely that the documents are ‘roughly the same’”. Likewise, the “containment” of “A in B is a number between 0 and 1 that, when close to 1, indicates that A is

‘roughly contained’ within B”. To compute the resemblance or containment, it is claimed, requires only a “sketch” of the document of “a few hundred bytes”.<sup>10</sup> The first step in this process is to convert the entire document to plaintext, then to tokenize the document into words. Once this is done, the process is to “ $\omega$ -shingle” the document into unique subtokens of size  $\omega$ . The example that Broder et al. give for a  $\omega = 4$  example is:

```
(a, rose, is, a, rose, is, a, rose)
```

translates to the set:

```
{(a, rose, is, a), (rose, is, a, rose), (is, a, rose, is)}
```

This means that, for any supplied shingle size, “the resemblance  $r$  of two documents A and B is defined as:

$$r(A, B) = \frac{|S(A) \cap S(B)|}{|S(A) \cup S(B)|}$$

Essentially, this means: take the intersection of the size of A’s shingles and B’s shingles and then divide it by the union size of A’s shingles and B’s shingles. The “containment” measure is calculated by taking the intersection of the size of A’s shingles and B’s shingles and then dividing it by the size of A’s shingles. (This is called Jaccard similarity.)

This method has a drawback. As the size of the compared documents grows in length, there is increasing likelihood that shingles will accidentally be shared between dissimilar documents. Thus, to ensure against this disadvantage it is important to select a  $\omega$ -shingle length that is long enough to capture unique runs. It is also true that Jaccard similarity is biased against documents of differing set sizes. However, this does not matter for our use case: if the document sizes are very different, it is very likely that they have drifted anyway.

By contrast, the formula for Jaccard containment, where we are measuring whether one document is contained within another is:

$$r(A, B) = \frac{|S(A) \cap S(B)|}{|S(A)|}$$

Jaccard *containment* may actually be more useful than Jaccard similarity for the task at hand (content drift detection). If we assume that comments and other user-generated content is only likely to grow over time, around the main document, then seeing whether the first snapshot is contained within subsequently retrieved pages may be a good way to work around this.

A variant of Jaccard similarity/containment can be obtained by a system called Lazo, that uses MinHash signatures to estimate JS/JC in  $o(n)$  time.<sup>[11](#)</sup>

An implementation of Jaccard similarity and Jaccard containment can be found at <https://github.com/MartinPaulEve/content-drift/blob/main/engines/jaccard.py>.

## Euclidean Distance

Euclidean distance refers to the distance in multi-dimensional space between two vector representations of the documents that one wishes to measure. The formula for calculating the Euclidean distance is  $d_{\text{Euc}}(d_1, d_2) = [(d_1 - d_2) \cdot (d_1 - d_2)]^{1/2}$ . In other words, the Euclidean distance between two documents is “the root of square differences between the respective coordinates of  $d_1$  and  $d_2$ ”.<sup>[12](#)</sup> Unlike Jaccard similarity, the Euclidean distance does not produce a single number between 0 and 1 to indicate document similarity. It instead provides a distance that may scale in proportion to the size of the documents in question. As Poornima Muthukumar puts it: “Smaller documents will result in vectors with a smaller magnitude and larger documents will result in vectors with larger magnitude as the magnitude of the vector is directly proportional to the number of words in the document, thereby making the overall distance larger”.<sup>[13](#)</sup>



The vector representation stage consists of creating an ordered matrix of token counts for each document and then converting it to an array (in Python we can use a CountVectorizer from scikit-learn to create this).

An implementation of Euclidean distance calculation can be found at <https://github.com/MartinPaulEve/content-drift/blob/main/engines/euclidean.py>.

## Cosine Similarity

Cosine similarity is a widely-used method for measuring the similarity between documents. It takes two vectors and calculates the cosine of the angle between the documents in multi-dimensional space. Similarly to Jaccard similarity, cosine similarity expresses similitude on a defined scale from -1 to 1, with 1 indicating that the documents are identical and -1 showing completely different documents.

The first step for calculating cosine similarity is to create a vector list of tf-idf (term frequency – inverse document frequency) for the whole text. This consists of taking the term frequency (the number of times the term occurs in the document divided by the total number of terms in the document) and multiplying it by the logarithmically scaled inverse fraction of the number of documents that contain the word (this can be derived by dividing the total number of documents by the number of documents containing the term, and then taking the logarithm of that count).

The next step is to calculate the cosine similarity. In Python, the scikit-learn package provides a cosine\_similarity function under the pairwise module. The formula that is used is:

$$K(X, Y) = \langle X, Y \rangle / (||X|| * ||Y||)$$

where X is an array of the document to be measured and Y is all the samples to measure (the total document set). This means take the inner product of X and Y and divide it by the norm of X multiplied by the norm of Y.

An important part of the tf-idf vectorization process is stopwords elimination. Without this, the document scores for long documents will end up being extremely and incorrectly high.



An implementation of the cosine coefficient can be found at:

<https://github.com/MartinPaulEve/content-drift/blob/main/engines/cosine.py>

## Word Embeddings

“Word embeddings” refer to dense vectors of words, with similar words sharing similar vectors. We can calculate document similarity by taking the average of the embeddings of all words in each document and then taking the cosine similarity between these averages. Using this approach requires a pre-trained word vector model, such as [Google’s pre-trained Word2Vec model](#).

An implementation of this word embedding approach can be found at:

[https://github.com/MartinPaulEve/content-drift/blob/main/engines/word\\_embeddings.py](https://github.com/MartinPaulEve/content-drift/blob/main/engines/word_embeddings.py)

## Sentence and Document Embeddings

Similar to Word Embeddings except that the vectorisation comparison takes place at the sentence and document level. We can use pre-trained systems such as BERT (Bidirectional Encoder Representations from Transformers) to generate these embeddings.

This method, as with Word Embeddings, has the disadvantage of having to download a pre-trained model. As we shall see, it is also certainly slower on some of our tests than other approaches, despite being a more “modern” method.

An implementation of this sentence embedding approach can be found at:

[https://github.com/MartinPaulEve/content-drift/blob/main/engines/sentence\\_embeddings\\_bert.py](https://github.com/MartinPaulEve/content-drift/blob/main/engines/sentence_embeddings_bert.py)

## Siamese Networks

These are individually trained neural network architectures that learn a similarity function. In other words, “they can be used to directly learn the similarity between two documents based on their content”.<sup>14</sup> These models require a dataset for training and do not appear to be an easy way to implement generic document comparisons.

## **Transformer Models (e.g. GPT)**

Transformer models, the basis of ChatGPT and other text-generation models, can be used for document similarity and can often capture the semantics of natural language more effectively than the earlier methods in this document. Fine tuning the pre-trained models for specific datasets or tasks can lead to highly accurate results. However, these models require API access and a remote network call, causing latency and expense.

Theoretically, models like this should be better at ignoring synonym substitution: for example, “the document was changed” vs. “the document was altered”. Whereas conventional approaches would see these as “substantial changes” because the underlying lexical formulation has changed, the transformer approach understands the contextual usage of these terms and would not see them as a major difference. The question then becomes, once more, a social choice: in the detection of content drift, are we interested in marking these types of changes as “drift”, or not? The OpenAI system also cannot handle very long prompts with token length greater than 8192 tokens, which a web page may easily exceed.

An implementation of this transformer approach using the OpenAI API can be found at:  
[https://github.com/MartinPaulEve/content-drift/blob/main/engines/transformer\\_openai.py](https://github.com/MartinPaulEve/content-drift/blob/main/engines/transformer_openai.py).

## **Neural Network-Based Clustering**

Techniques based on neural networks can cluster similar documents together in multi-dimensional space. This is a complex system to create from scratch, but pre-trained models/libraries such as HDBSCAN can help.

# **Evaluation of Approaches**

## **The Document Test Suite**

There are six document tests within our document test suite that each measure a different aspect of document similarity accuracy and performance.

- The identical test measures the ability of an algorithm to detect identical documents.

- The long data test measures the ability of an algorithm to parse two approximately 8,000 word documents that are entirely different from each other.
- The paragraph test replaces an entire paragraph of a document, but nothing more, resulting in an approximate 25% change to the document. This is a short, four-paragraph, document.
- The synonym replacement test replaces a substantial number of words in the document with synonyms. The test is approximately 400 words in length.
- The typo test introduces a set of small typos into a 400-word document.
- The containment test has document a contained in document b, with additional surplus “internet comments” at the bottom of the second document.

## Overall Results

These results were obtained by running the all-tests command on my [evaluation application](#). A note: the application tries to measure the timing of imports, which are handled locally (and dynamically in the test suite). However, it is possible that some third party libraries or Python itself implements caching so as not to reimport a module. Therefore, runtimes on the first test, in which the documents are identical may be inflated.

In the table below:

- Algo = algorithm
- t = time in seconds
- r = result
- ID = identical / Long = long data / para = paragraph / syno = synonym replacement / typo = typos
- Jaccard = Jaccard Similarity / JaccC = Jaccard Containment / Euclid = Euclidean Distance / Cos = tf-idf Cosine Similarity / WordE = Word Embedding / SentE = Sentence Embedding / OpenAI = Transformer OpenAI

| Algo    | ID_t   | ID_r  | long_t | long_r | para_t | para_r | syno_t | syno_r | typo_t | typo_r |
|---------|--------|-------|--------|--------|--------|--------|--------|--------|--------|--------|
| Jaccard | 0.001  | 1.0   | 0.018  | 0.028  | 0.002  | 0.703  | 0.001  | 0.677  | 0.001  | 0.991  |
| JaccC   | 0.001  | 1.0   | 0.017  | 0.052  | 0.001  | 0.734  | 0.001  | 0.802  | 0.001  | 0.996  |
| Euclid  | 0.299  | 0.0   | 0.007  | 257    | 0.001  | 15.588 | 0.001  | 7.874  | 0.001  | 1.414  |
| Cos     | 0.011  | 0.999 | 0.008  | 0.118  | 0.002  | 0.843  | 0.001  | 0.901  | 0.001  | 0.996  |
| WordE   | 13.293 | 0.999 | 14.918 | 0.911  | 14.460 | 0.983  | 14.333 | 0.993  | 14.144 | 0.999  |
| SentE   | 2.647  | 1.00  | 1.103  | 0.321  | 1.104  | 0.991  | 1.117  | 0.978  | 1.095  | 0.999  |
| OpenAI  | 0.638  | 0.999 | N/A    | N/A    | 0.489  | 0.979  | 2.363  | 0.974  | 0.513  | 0.999  |

## Jaccard Similarity

The Jaccard similarity algorithm was among the best performing of the tested algorithms. The algorithm consistently ran within acceptable timings even on longer documents.

On the identical document test, this algorithm returned the correct value of 1.0 in 0.001 seconds. The long document test took a greater, but still very small, time period of 0.018 seconds, while also scoring these (very different) documents as far apart, with a score of 0.028. The paragraph change document test was fast at 0.002 seconds but this was one of the only algorithms to get close to the actual percentage change of the document at a score of 0.703 (approximately  $\frac{1}{4}$  of the document has changed). The Jaccard test fares badly on synonym detection, and while it could perform the check quickly, it has no contextual vector awareness of words that mean the same thing, so it fails to understand how the documents might be the same, with a similarity score of 0.677. As noted in the

conclusion below, this may actually be a strength of the Jaccard approach. Finally, introducing small typos threw this model off more than other approaches, although not by much, with a score of 0.991.

Jaccard Containment is also a special case that fares particularly well on a test not listed in the table above: the containment test. This document is a chunk of the same text, but with one copy having an “internet comments”-like section appended to the end. Evaluating for document A in document B scores 1.0 in Jaccard containment (i.e. a perfect copy of document A exists within document B).

In short, depending on the definition of “content drift”, Jaccard Similarity and especially Jaccard Containment remain potentially powerful tools for evaluating document similarities and containment.

## **Euclidean Distance**

Euclidean distance is more challenging to compare to the other methods since it doesn’t produce results on a simple scale, but instead gives larger distances for documents that are further apart in multi-dimensional space. That said, the algorithm generally performs well, with a possible first-load performance penalty exhibited on the identical test. (The two imports that are run are: “from sklearn.feature\_extraction.text import CountVectorizer” and “from scipy.spatial.distance import euclidean”.) The identical test produced the expected outcome of “0.0” for identical documents. The long document test, on the other hand, produced a result of 257; a relatively high number indicating substantial difference but with no scale measure to indicate how high a different measure can go. This algorithm ran quickly, even on this longer document. Again, the paragraph replacement indicated some level of difference with a score of 15.588, but without any indication as to how this number relates to the idea of a 25% change. The synonym replacement test indicated a substantial level of distance from only small, contextually insignificant, changes at a score of 7.874. And finally, the typo test indicated small changes.

The euclidean distance measure seems most useful when comparing multiple documents to each other, when you want to know which set of documents bear the most resemblance to each other. In such a scenario, the scale is taken as relative to the document set. It does not work so well for binary

comparisons, where the scale appears arbitrary. A further way of dealing with this is normalization of the vector using Euler's constant ( $1/\exp(\text{distance})$ ).

## **Cosine Similarity**

The tf-idf vectorization to cosine similarity algorithm appears to be a fast algorithm that can accurately detect lexical changes in documents of all kinds. The results for the identical test were 0.999 while the results for the long test with two very different documents yielded 0.118, accurately detecting the dissimilarity. Notably, this algorithm only performed well when English stopwords were removed (and, obviously, this would have to be changed to be locale specific). This stopword removal could prove problematic if working in a multilingual environment, such as the web, where it may be difficult to detect the language of a web page. Without the stop word removal, the long documents scored 0.918 similarity (a high score implying that the documents are similar). This method also fared relatively well on the paragraph comparison test, scoring 0.843 for a document with approximately 75% similar content.

## **Word Embeddings**

The results from the word embeddings approach are not promising. The first drawback to this method is that it requires a pre-trained Word2Vec model. The Google pre-trained model suggested above is 3.6GB when extracted, requiring significant disk space. Secondly, if the model is not preloaded, this approach is extremely slow, taking approximately 14 seconds(!) for most operations. Finally, this method did not produce good results. The long data test, with completely different texts, produced a high similarity score of 0.911 while the paragraph test, with 25% of content altered, scored 0.982.

Speculatively, reasons why this is failing to accurately delineate two very different documents could lie in the mean normalization of document content and the exclusion of words that were not in the pre-trained model (our sample test was two works of openly licensed literary criticism with specialist vocabulary). A similar score for the long text documents was obtained when using the glove.6B.300d pretrained model.

## **Sentence Embeddings**

This approach worked better than the word embeddings method detailed above. However, it was a particularly slow approach compared to the basic algorithms (though not compared to Word Embeddings). The identical test took a long 2.647 seconds to complete and none of the tests took less than a second, even when dealing with only 400 words. This approach also did not produce a sane result for the paragraph replacement similarity test, scoring a document with 25% change as 0.991.

## **Transformer Models (OpenAI)**

These models were relatively slow and expensive because they require an API call per similarity check. The timing is arbitrary and depends upon network conditions at the time of the call, with timings ranging from 0.489 to 2.363 seconds. This model fared badly, again, on the paragraph test, claiming that a document with a 25% substitution exhibited 0.979 similarity. However, these models seemed to have a greater awareness of synonyms as not changing the meaning of texts, scoring these semantically equivalent formulations as highly similar. See the overall conclusion below for the social choices that must be made around synonyms.

## **Notes on Preprocessing**

The biggest decision when preprocessing is whether to include a JavaScript engine or not. Some academic journal articles, one of the prime targets of content drift monitoring, will not load without JavaScript shell processing. If a full JavaScript engine is required, the optimal solution is to use headless Chrome via Puppeteer or similar and use the `-dump-html` command line flag to dump the HTML once JavaScript has loaded.

As noted earlier the decisions around JavaScript parsing are based on a series of different factors:

- Some pages will not load without JavaScript
- JavaScript parsing adds substantial complexity and processing overhead to preprocessing
- Comments and other dynamic content, often dependent on JavaScript, can adversely affect content drift detection (this can be overcome by using containment measures, as above under Jaccard Containment)



After this, parsing the HTML can be achieved via several libraries. For instance, the HTML2Text library (GPL licensed) does a decent job, while the inscriptis Python library is capable and generates layout-sensitive plaintext content.

## Conclusions

Despite more modern document similarity methods existing, the Jaccard similarity algorithm (or Jaccard containment) or even basic cosine similarity remain the most effective tools for content drift detection (which is a different task to mere document similarity detection). They rely on no hefty external models and can be calculated quickly for a document pair. They evaluate lexical similarity, as opposed to any kind of synonym similarity or semantic context. To demonstrate this, consider the two sentences: “The jar is empty” and “There is nothing in the jar”. While a human might say that these sentences mean the same thing, the lexical similarity measure would determine that these are very different. The question is: in the context of “content drift” of web pages with a DOI, are we interested in lexical difference or semantic difference? I contend that the answer is “the former” (lexical difference), because when people are citing exact phrases, its replacement by a semantically similar formulation erases the validity of the citation. For this reason, older methods that measure lexical similarity or containment may be a more appropriate tool for content drift detection.

It is also true that more basic approaches, such as feature detection (DOI, title) on a page may prove a better measure of content drift detection than textual similarity changes. That is to say: it may be, in this case, that the simpler methods are the best for this particular task.

1. Shawn M. Jones and others, ‘Scholarly Context Adrift: Three out of Four URI References Lead to Changed Content’, *PLOS ONE*, 11.12 (2016), p. e0167475, doi:[10.1371/journal.pone.0167475](https://doi.org/10.1371/journal.pone.0167475). The term has also been used in a different way to refer to how content changes over time in social media contexts (e.g. because camera lenses become sharper) Dmitry Baranchuk and others, ‘DeDrift: Robust Similarity Search under Content Drift’ (arXiv, 2023), doi:[10.48550/arXiv.2308.02752](https://doi.org/10.48550/arXiv.2308.02752). ↩

2. Junghoo Cho and Hector Garcia-Molina, 'The Evolution of the Web and Implications for an Incremental Crawler', in *VLDB '00: Proceedings of the 26th International Conference on Very Large Data Bases* (presented at the VLDB00: Very Large Data Bases, Morgan Kaufmann Publishers Inc., 2000), pp. 200–209 (pp. 201–2). ↩
3. Cho and Garcia-Molina, 'The Evolution of the Web and Implications for an Incremental Crawler', p. 202. See also Junghoo Cho and Hector Garcia-Molina, 'Estimating Frequency of Change', *ACM Trans. Internet Technol.*, 3.3 (2003), pp. 256–90, doi:[10.1145/857166.857170](https://doi.org/10.1145/857166.857170). ↩
4. Fred Douglass and others, 'Rate of Change and Other Metrics: A Live Study of the World Wide Web', in *Proceedings of the USENIX Symposium on Internet Technologies and Systems* (presented at the USENIX Symposium on Internet Technologies and Systems, 1997). ↩
5. John Bowers, Jonathan Zittrain, and Clare Stanton, 'The Paper of Record Meets an Ephemeral Web: An Examination of Linkrot and Content Drift within The New York Times', *SSRN Electronic Journal*, 2021, p. 13 (p. 4), doi:[10.2139/ssrn.3833133](https://doi.org/10.2139/ssrn.3833133). ↩
6. For a good overview of distance measures for document similarity, see Mahmoud Harmouch, '17 Types of Similarity and Dissimilarity Measures Used in Data Science', Medium, 2021 <https://towardsdatascience.com/17-types-of-similarity-and-dissimilarity-measures-used-in-data-science-3eb914d2681> [accessed 25 July 2024]. ↩
7. Kavitha Karun A, Mintu Philip, and K Lubna, 'Comparative Analysis of Similarity Measures in Document Clustering', in *2013 International Conference on Green Computing, Communication and Conservation of Energy (ICGCE)* (presented at the 2013 International Conference on Green Computing, Communication and Conservation of Energy (ICGCE), IEEE, 2013), pp. 857–60, doi:[10.1109/ICGCE.2013.6823554](https://doi.org/10.1109/ICGCE.2013.6823554). ↩
8. Dennis Fetterly and others, 'A Large-Scale Study of the Evolution of Web Pages', in *WWW '03: Proceedings of the 12th International Conference on World Wide Web* (presented at the 12th international conference on World Wide Web, Association for Computing Machinery, 2003),

- pp. 669–78; Andrei Z. Broder and others, ‘Syntactic Clustering of the Web’, *Computer Networks and ISDN Systems*, Papers from the Sixth International World Wide Web Conference, 29.8 (1997), pp. 1157–66, doi:[10.1016/S0169-7552\(97\)00031-7](https://doi.org/10.1016/S0169-7552(97)00031-7). ↵
9. In turn, this is based on the work of Nevin Heintze, ‘Scalable Document Fingerprinting (Extended Abstract)’, in *Proceedings of the 2nd USENIX Workshop on Electronic Commerce* (presented at the 2nd USENIX Workshop on Electronic Commerce, 1996). ↵
10. Broder and others, p. 1158. ↵
11. Raul Castro Fernandez and others, ‘Lazo: A Cardinality-Based Method for Coupled Estimation of Jaccard Similarity and Containment’, in 2019 IEEE 35th International Conference on Data Engineering (ICDE), 2019, pp. 1190–1201, doi:[10.1109/ICDE.2019.00109](https://doi.org/10.1109/ICDE.2019.00109). ↵
12. Yung-Shen Lin, Jung-Yi Jiang, and Shie-Jue Lee, ‘A Similarity Measure for Text Classification and Clustering’, *IEEE Transactions on Knowledge and Data Engineering*, 26.7 (2014), pp. 1575–90 (p. 1576), doi:[10.1109/TKDE.2013.19](https://doi.org/10.1109/TKDE.2013.19). ↵
13. Poornima Muthukumar, ‘Evaluating Methods for Calculating Document Similarity’, KDnuggets, 2023 <https://www.kdnuggets.com/evaluating-methods-for-calculating-document-similarity> [accessed 17 July 2024]. ↵
14. See Rany ElHousieny, ‘Document Similarity with Examples in Python’, 2024 <https://www.linkedin.com/pulse/document-similarity-examples-python-rany-elhousieny-phd%E1%B4%AC%E1%B4%AE%E1%B4%B0-0i5lc> [accessed 19 July 2024], to which much of this document is indebted. ↵